



Getting Started & the Python Model

Python Mastery — Part 1: Foundations · Week 1

Rathinam Trainers & Consultants

Live on Microsoft Teams · recorded



Welcome

How this course works

The Python Mastery Series

- ▶ **Part 1 — Foundations** (you are here) · 16 weeks
- ▶ Parts 2–4: Intermediate & OOP → Advanced → Expert
- ▶ From "never written a line" to confident, idiomatic Python

This part: **the core language + standard library**, ending in a real CLI app.

How every week runs

- ▶ **2 hours, live, recorded** — same time each week
- ▶ I **teach** the concept, then **demo it live** on screen
- ▶ You **practice after** from the recording + the lab guide
- ▶ A weekly **lab** rebuilds what the demo showed, then extends it

The recording *is* part of the courseware — scrub back any time.

The loop, every week

```
concept    -> live demo    -> your lab    -> checkpoint  
(me)       (me, on        (you,        (you submit,  
            screen)       after)         I confirm)
```

No in-session typing by you — watch now, build after.

What you need

- ▶ Any laptop you can **install software** on (Windows / macOS / Linux)
- ▶ **No prior programming** — absolute beginners welcome
- ▶ Coming from Java / C# / JS / Go? You'll move faster; the demos will point out where Python differs

Today's goal

By the end of this session you'll understand how to:

- ▶ Install **Python 3.14** and open the **new REPL**
- ▶ Run a `.py` file **two ways**
- ▶ Say in one sentence what *"the interpreter runs bytecode"* means
- ▶ Scaffold a project with `uv` and tidy it with **Ruff**

What Python is

And where 3.14 fits

What is Python?

- ▶ A **high-level, general-purpose** language — readable, batteries-included
- ▶ You write **source code**; you don't manage memory or compile by hand
- ▶ **CPython** is the standard build — the one we install and run
- ▶ Used for scripting, web, data, automation, AI — one language, many doors

Python 3.14 — current stable

- ▶ **3.14.0** released **Oct 2025**; **3.14.6** is current stable (Jun 2026)
- ▶ We pin the whole course to **3.14** so every demo reproduces
- ▶ Headline 3.14 features we'll touch in Part 1:
 - ▶ A much-improved **interactive REPL**
 - ▶ **t-strings** (PEP 750) — a peek in Week 3
 - ▶ Friendlier **exception** syntax — Week 11

How Python runs your code

your source		Python		the CPython
hello.py	----->	compiles to	----->	interpreter loop
(text you	(auto)	bytecode	runs	executes it
write)		(.pyc, internal)		step by step

- ▶ You never do this by hand — it happens when you run the file
- ▶ **One sentence:** *Python turns your text into bytecode, then the interpreter runs that bytecode.*

REPL vs script — two ways to run

REPL	Script
Type a line, see it now	Save a <code>.py</code> , run it later
For exploring & testing	For keeping & repeating
Forgets when you close it	Lives on disk, re-runnable

You'll use **both** every day. Today: meet both.

Let's open Python and talk to it.

Watch this — Demo 1

REPL tour + your first script

- ▶ Launch the **3.14 REPL** — colours & autocomplete
- ▶ Evaluate `2 + 2`, a string, `print(...)`
- ▶ `import ma` + **Tab** → it completes to `math`
- ▶ `help(len)`
- ▶ Write `hello.py`, run it `python hello.py`, then `python -m hello`

The new 3.14 REPL

```
>>> 2 + 2
4
>>> "Python".upper()
'PYTHON'
>>> import ma      # press Tab
import math
>>> help(len)
```

- ▶ **Syntax highlighting** on by default
- ▶ **Tab** completes import names
- ▶ `exit()` or Ctrl-D to leave

One file, two ways to run

```
$ python hello.py  
Hello, Python 3.14  
$ python -m hello  
Hello, Python 3.14
```

- ▶ `python hello.py` → run **this file** by path
- ▶ `python -m hello` → run the **module** named `hello`
- ▶ Same output — the `-m` form matters later for packages

What you just saw

- ▶ The REPL **answers instantly** — great for trying things
- ▶ **Tab** completes imports; `help()` reads the docs to you
- ▶ A script is **saved code** you run again and again
- ▶ `python file.py` and `python -m file` both ran it

The modern toolchain

uv and Ruff — from day one

Why tools on day one?

- ▶ Real projects are **not** one loose `.py` file
- ▶ Two everyday needs, even for beginners:
 - ▶ **Isolation** — each project gets its own clean setup
 - ▶ **Tidy code** — consistent formatting + early mistake catching
- ▶ We start the good habits now so they're invisible later

uv — the project manager

- ▶ One fast tool (from **Astral**) for:
 - ▶ Installing **Python** itself — `uv python install 3.14`
 - ▶ Scaffolding a **project** — `uv init`
 - ▶ Running it — `uv run`
 - ▶ (Later) dependencies & lockfiles — Week 10
- ▶ Replaces a pile of older tools with one command

Ruff — format & lint

- ▶ Also from **Astral**, extremely fast
- ▶ `ruff format` → consistent style, automatically
- ▶ `ruff check` → flags likely mistakes & messy code
- ▶ Run it with no install: `uvx ruff format`
- ▶ Free, open-source — like everything in Part 1

Let's turn a loose idea into a real project.

Watch this — Demo 2

`uv init` project + Ruff

- ▶ `uv init taskapp` → a real project appears
- ▶ Tour `pyproject.toml`, `main.py`, `README.md`, `.python-version`
- ▶ Run it: `uv run main.py`
- ▶ Paste messy code → `ruff format` fixes it
- ▶ `ruff check` flags a problem → fix it

What `uv init` creates

```
taskapp/  
├── .python-version      # pins Python 3.14  
├── README.md  
├── main.py              # def main(): print("Hello...")  
└── pyproject.toml       # project metadata
```

```
$ uv run main.py  
Hello from taskapp!
```

This `taskapp` is the **seed of your capstone**.

Ruff in action

```
$ uvx ruff format  
1 file reformatted
```

```
$ uvx ruff check  
All checks passed!
```

- ▶ `format` rewrites spacing, quotes, line length for you
- ▶ `check` reports issues; `ruff check --fix` fixes the safe ones

What you just saw

- ▶ `uv init` made a **complete, runnable project** in seconds
- ▶ `uv run` ran it inside its own managed environment
- ▶ **Ruff** cleaned the formatting and verified the code
- ▶ `taskapp` is now the project we grow for 16 weeks

Q&A + install help

Common pitfalls per OS

Install gotchas

- ▶ **Windows:** use the Microsoft Store / Python Install Manager; or tick "**Add Python to PATH**" in the classic installer
- ▶ **macOS:** use the `.pkg` from python.org, or `uv python install 3.14`
- ▶ **Linux:** your distro's Python may be old — `uv python install 3.14`
- ▶ Simplest cross-OS path: install `uv` first, let it fetch Python

python vs python3 vs py

- ▶ macOS / Linux: often `python3` (and `python3.14`)
- ▶ Windows: the `py` launcher, e.g. `py -3.14`
- ▶ Inside a `uv` project: just `uv run ...` — no guessing
- ▶ If a command "isn't found", it's almost always a **PATH** issue

Wrap-up

This week's homework

Lab 1 — set up & prove your toolchain

1. Install **Python 3.14**, **uv**, **Ruff**, **VS Code**
2. In the **REPL**: evaluate three expressions
3. Write `hello.py` ; run it **both ways**
4. `uv init taskapp` ; run it; **format** with Ruff
5. **Submit**: screenshots of the REPL + script, and your formatted project

Mirror **Demo 1** then **Demo 2** from the recording.

Next week

- ▶ **Week 2 — Numbers, Variables & Expressions**
- ▶ We'll use the REPL and `taskapp` you set up today
- ▶ Bring your working environment — everything builds on it

See you in the lab.

Questions?